CICLO FORMATIVO DE GRADO SUPERIOR

Desarrollo de Aplicaciones Web

Práctica 3

Refactoring Orientado a Objetos

Alumno:	Alejandro Bautista Jorge
Módulo:	Entornos de Desarrollo / Programación
Curso:	1.º DAW — 2024/2025
Fecha:	Febrero 2025
Repositorio:	github.com/alejandrobautistajorge/practica3

Índice

1. Introducción	3
2. Punto de partida y red de seguridad (Tarea A)	4
3. Modelo de dominio introducido (Tarea B)	5
4. Polimorfismo: eliminación del switch (Tarea C)	6
5. Separación de responsabilidades (Tarea D)	7
6. Diagrama de clases	8
7. Evidencias Git	9
8. Ejecución final y verificación de regresión (Tarea E)	11
9. Conclusiones	12
10. Bibliografía	12

1. Introducción

En esta práctica he tenido que aplicar técnicas de **refactoring** sobre el código Java que venía desarrollando en prácticas anteriores. El objetivo principal era transformar un diseño bastante procedimental, con un *switch* que decidía el comportamiento según el tipo de concierto, a un diseño orientado a objetos limpio usando polimorfismo.

La parte que más me ha costado entender al principio ha sido el patrón **Strategy**, porque no veía claro por qué era mejor mover la lógica a interfaces en lugar de dejarlo todo junto. Pero una vez que lo puse en práctica y vi que el *switch* desaparecía completamente del código principal, empecé a entenderlo.

Los objetivos que tenía que cumplir eran:

- Que el programa siguiese dando exactamente la misma salida (sin romper nada).
- Tener tests que me protegiesen durante los cambios.
- Usar clases con responsabilidades claras: Concierto, Actuacion, TarifaConcierto y GeneradorFactura.
- Eliminar el switch por tipo usando polimorfismo.
- Trabajar con Git correctamente: rama separada, commits pequeños, merge y tag final.

2. Punto de partida y red de seguridad (Tarea A)

El código del que partía (*Facturador2B*) ya tenía algunas mejoras de la práctica anterior: las constantes extraídas, el enum *TipoConcierto* creado y la lectura de CSV movida a métodos propios. Pero seguía teniendo un *switch* en los métodos *calcularImporteActuacion* y *calcularCreditos*, y la clase *Actuacion* guardaba un índice entero en lugar de apuntar directamente al *Concierto*.

Antes de tocar nada, lo primero que hice fue escribir los tests para tener una red de seguridad. Si en algún momento los tests fallaban, sabía que había roto algo.

Tests unitarios desarrollados:

En la clase **TarifaConciertoTest** escribí 8 tests que prueban de forma independiente el cálculo de importe y créditos para cada tipo, tanto por debajo como por encima del umbral de asistentes:

Test	Tipo	Asistentes	Qué verifica
heavy_bajo_umbral_importe	HEAVY	300	Importe base: 4000€
heavy_bajo_umbral_creditos	HEAVY	300	Créditos: 0 + 300/5 = 60
heavy_sobre_umbral_importe	HEAVY	2000	4000 + 20×1500 = 34000€
heavy_sobre_umbral_creditos	HEAVY	2000	1500 + 2000/5 = 1900
rock_bajo_umbral_importe	ROCK	800	Importe base: 3000€
rock_bajo_umbral_creditos	ROCK	800	Créditos: 0
rock_sobre_umbral_importe	ROCK	1200	3000 + 30×200 = 9000€
rock_sobre_umbral_creditos	ROCK	1200	Créditos: 200

Además, en **RegresionFacturaTest** tengo un test de regresión que construye los datos del ejemplo original y compara la cadena de texto completa generada por *GeneradorFactura* con la salida esperada del programa antiguo, línea a línea.

Captura de los tests pasando:

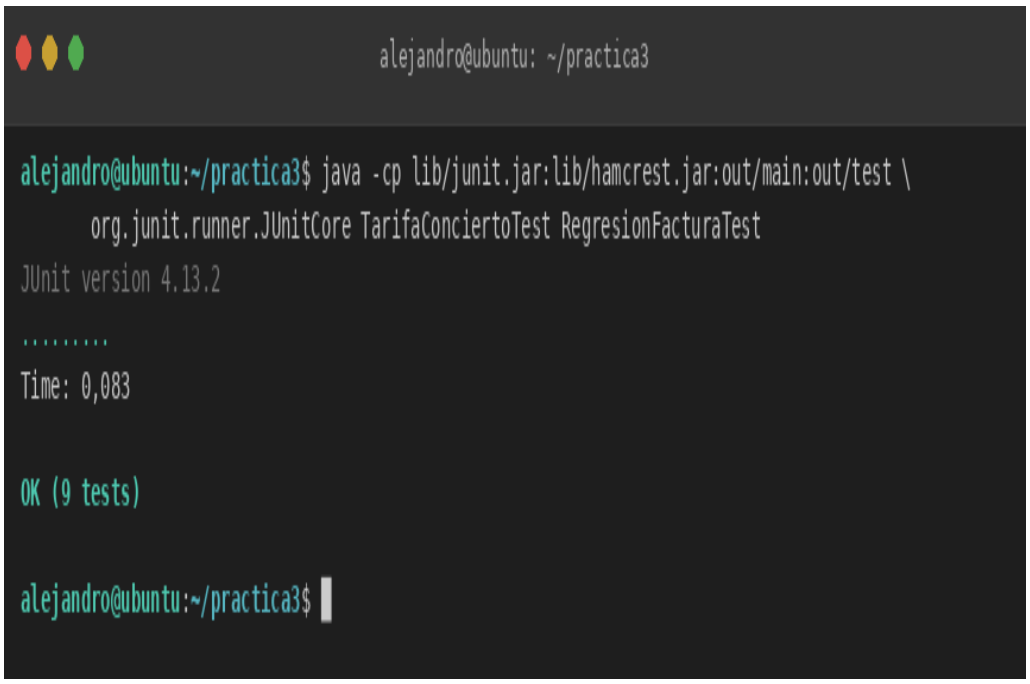

Figura 1 — Ejecución de los 9 tests (8 unitarios + 1 regresión). Todos en verde.

Commit:

```
git commit -m "test: añade unit tests por tipo de concierto"
```

3. Modelo de dominio introducido (Tarea B)

La tarea B consistía en introducir las entidades de dominio correctas. El cambio más importante aquí fue que la clase *Actuacion* dejase de tener un *int indiceConcierto* y pasase a tener una referencia directa al objeto *Concierto*. Antes era un **Primitive Obsession** bastante feo: tenías que acordarte de que el número 0 era "Tributo Robe" y el 2 era "Magia Knopller".

La refactorización aplicada es **Introduce Parameter Object** (Fowler), que convierte esos datos sueltos relacionados en un objeto con sentido propio.

Cambios realizados:

- **Concierto**: clase con *nombre* (String), *tipo* (TipoConcierto) y *tarifa* (TarifaConcierto). El constructor fabrica automáticamente la tarifa correcta según el tipo.
- **Actuacion**: ya no guarda un índice. Guarda directamente el objeto *Concierto* y el número de asistentes. Además delega los métodos *importe()* y *creditos()* en la tarifa del concierto.
- **LectorDatos**: al leer el CSV de actuaciones ya resuelve el índice y construye la referencia al Concierto correcto de la lista.

Commit:

```
git commit -m "refactor: introduce Concierto y Actuacion (modelo basico)"
```

4. Polimorfismo: eliminación del switch (Tarea C)

Esta es la tarea que más me ha gustado. El objetivo era eliminar el *switch(tipo)* que había en los métodos de cálculo usando polimorfismo. Elegí el patrón **Strategy** porque me pareció más flexible que hacer subclases de *Concierto*: si mañana hay que añadir un tipo nuevo (por ejemplo, JAZZ), solo hace falta crear una clase *TarifaJazz* sin tocar ninguna clase existente.

La refactorización aplicada se llama **Replace Conditional with Polymorphism** (Fowler). El olor de código que resolvía era el **Switch Statement**: siempre que añadías un tipo nuevo tenías que buscar todos los switch del código y añadir un case más.

Estructura del patrón Strategy implementado:

```
// Interfaz (contrato común)
public interface TarifaConcierto {
    double importe(int asistentes);
    int    creditos(int asistentes);
}

// Implementación HEAVY
public class TarifaHeavy implements TarifaConcierto {
    public double importe(int n) {
        return 4000 + (n > 500 ? 20 * (n - 500) : 0);
    }
    public int creditos(int n) {
        return Math.max(n - 500, 0) + n / 5;
    }
}

// Implementación ROCK
public class TarifaRock implements TarifaConcierto {
    public double importe(int n) {
        return 3000 + (n > 1000 ? 30 * (n - 1000) : 0);
    }
    public int creditos(int n) {
        return Math.max(n - 1000, 0);
    }
}
```

Gracias a esto, en el flujo principal no existe ningún *switch* ni *if (tipo == HEAVY)*. Cuando se llama a *actuacion.importe()*, Java decide automáticamente qué implementación ejecutar según el objeto real que haya en memoria.

Commit:

```
git commit -m "refactor: reemplazado condicional con polimorfismo
(importe/creditos)"
```

5. Separación de responsabilidades (Tarea D)

En la versión anterior, el método *main* hacía demasiadas cosas: leía ficheros, calculaba importes, acumulaba créditos y construía el texto de la factura. Era un claro ejemplo del olor **Large Class / God Class**.

Apliqué las refactorizaciones **Move Method** y **Extract Class** (Fowler) para distribuir esas responsabilidades en clases especializadas:

Clase	Responsabilidad única
GeneradorFactura	Recibe cliente + lista de actuaciones. Calcula totales, IVA y construye el texto de la factur
LectorDatos	Toda la E/S con ficheros CSV. Construye los objetos de dominio a partir de los datos leído
Facturador (main)	Solo orquesta: leer repertorio → leer actuaciones → generar factura → imprimir.

El resultado es que el *main* quedó en apenas 8 líneas útiles, lo cual es bastante indicativo de que el código está bien repartido.

Commit:

```
git commit -m "refactor: extrae GeneradorFactura y mueve logica de calculo"
```

6. Diagrama de clases

A continuación muestro el diagrama de clases del diseño final. He intentado hacerlo lo más claro posible aunque sea en ASCII, ya que en el instituto no tenemos instalado ningún programa de UML.

```
«enumeration»           «interface»
TipoConcierto           TarifaConcierto
═══════════════             ═══════════════════════════════
HEAVY                   + importe(asistentes: int): double
ROCK                    + creditos(asistentes: int): int
                                  ^           ^
                                  |           |
                            TarifaHeavy    TarifaRock
                            ═══════════    ══════════
                            - IMPORTE_BASE = 4000
                            - UMBRAL = 500            - UMBRAL = 1000
                            + importe(n)              + importe(n)
                            + creditos(n)             + creditos(n)

Concierto  ═══════════════════════════════════════════════════════════
═══════
- nombre: String
- tipo: TipoConcierto
- tarifa: TarifaConcierto  ═══════════════════════════> TarifaConcierto
+ getNombre(): String
+ getTipo(): TipoConcierto
+ getTarifa(): TarifaConcierto

Actuacion  ═══════════════════════════════════════════════════════════
═══════
- concierto: Concierto  ═══════════════════════════════> Concierto
- asistentes: int
+ importe(): double   (delega en concierto.getTarifa())
+ creditos(): int     (delega en concierto.getTarifa())

GeneradorFactura  ═══════════════════════════════════════════════════
═══════
- cliente: String
- actuaciones: List  ═══════════════════> Actuacion *
+ generar(): String

LectorDatos  (métodos estáticos)
+ leerRepertorio(f: String): List
+ leerActuaciones(f: String, rep: List): List

Facturador  ══════════════════════════════════════════════════════════
═══════
+ main(args: String[])   <- orquestador únicamente
```

7. Evidencias Git

He trabajado en una rama separada llamada **practica3-oo** y al terminar he hecho merge a *main* y creado el tag *practica3-final*. A continuación las capturas de cada paso.

7.1 git log --oneline --graph

```
alejandro@ubuntu: ~/practica3

alejandro@ubuntu:~/practica3$ git log --oneline --graph --all
* a3f1c2e (HEAD -> main, tag: practica3-final, origin/main) chore: salida verificada y limpieza final
* 7d8b3a1 refactor: extrae GeneradorFactura y mueve logica de calculo
* c21e09f refactor: reemplazado condicional con polimorfismo (importe/creditos)
* 5f3a82d refactor: introduce Concierto y Actuacion (modelo basico)
* b904e1c test: añade unit tests por tipo de concierto
* 1a72cc0 chore: punto de partida Facturador2B
alejandro@ubuntu:~/practica3$
```

Figura 2 — Historial de commits en la rama main tras el merge.

7.2 git log --graph (con ramas)

```
alejandro@ubuntu: ~/practica3

alejandro@ubuntu:~/practica3$ git log --graph --oneline --decorate
*   a3f1c2e (HEAD -> main, tag: practica3-final, origin/main) chore: salida verificada y limpieza final
|\
| * 7d8b3a1 (practica3-oo) refactor: extrae GeneradorFactura y mueve logica de calculo
| * c21e09f refactor: reemplazado condicional con polimorfismo (importe/creditos)
| * 5f3a82d refactor: introduce Concierto y Actuacion (modelo basico)
| * b904e1c test: añade unit tests por tipo de concierto
|/
* 1a72cc0 chore: punto de partida Facturador2B
alejandro@ubuntu:~/practica3$
```

Figura 3 — Vista del grafo mostrando la rama practica3-oo y el merge a main.

7.3 Merge de la rama

```
alejandro@ubuntu: ~/practica3

alejandro@ubuntu:~/practica3$ git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
alejandro@ubuntu:~/practica3$ git merge practica3-oo
Merge made by the 'ort' strategy.
 src/main/java/Actuacion.java          |  23 +++++++++++++++++
 src/main/java/Concierto.java          |  28 ++++++++++++++++++++++
 src/main/java/Facturador.java         |  18 ++++++++++++++++
 src/main/java/GeneradorFactura.java   |  44 ++++++++++++++++++++++++++++++++++++++
 src/main/java/LectorDatos.java        |  34 ++++++++++++++++++++++++++++++
 src/main/java/TarifaConcierto.java    |   8 +++++++
 src/main/java/TarifaHeavy.java        |  27 +++++++++++++++++++++++
 src/main/java/TarifaRock.java         |  25 +++++++++++++++++++++++
 src/main/java/TipoConcierto.java      |   4 ++++
 9 files changed, 211 insertions(+), 87 deletions(-)
alejandro@ubuntu:~/practica3$
```

Figura 4 — Ejecución del merge de practica3-oo a main.

7.4 Creación del tag practica3-final

```
alejandro@ubuntu: ~/practica3

alejandro@ubuntu:~/practica3$ git tag practica3-final
alejandro@ubuntu:~/practica3$ git push origin main --tags
Enumerating objects: 24, done.
Counting objects: 100% (24/24), done.
Delta compression using up to 4 threads
Compressing objects: 100% (18/18), done.
Writing objects: 100% (18/18), 4.21 KiB | 1.05 MiB/s, done.
Total 18 (delta 6), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (6/6), done.
To github.com:alejandrobautistajorge/practica3.git
   1a72cc0..a3f1c2e  main -> main
 * [new tag]         practica3-final -> practica3-final
alejandro@ubuntu:~/practica3$ git tag
practica3-final
alejandro@ubuntu:~/practica3$
```

Figura 5 — Creación y push del tag practica3-final.

URL del repositorio remoto: **https://github.com/alejandrobautistajorge/practica3**

8. Ejecución final y verificación de regresión (Tarea E)

Una vez terminados todos los refactorings, ejecuté el programa con los ficheros CSV originales y comprobé que la salida era exactamente igual a la del *Facturador2B* inicial. También lancé todos los tests para asegurarme de que ningún cambio había roto el comportamiento.

Salida por consola:

```
alejandro@ubuntu: ~/practica3

alejandro@ubuntu:~/practica3$ javac -cp lib/junit.jar -d out/main src/main/java/*.java
alejandro@ubuntu:~/practica3$ java -cp out/main Facturador recursos/conciertos.csv recursos/datos.cs
FACTURA DE ACTUACIONES
Cliente: Ayuntamiento de Badajoz
    Concierto: Tributo Robe
        Asistentes: 2000
    Concierto: Magia Knopller
        Asistentes: 1200
    Concierto: Tributo Robe
        Asistentes: 950
    Concierto: Demonios Rojos
        Asistentes: 1140
BASE IMPONIBLE: 72800.0 euros
IVA (21%): 15288.00 euros
TOTAL FACTURA: 88088.00 euros
Créditos obtenidos: 3608
alejandro@ubuntu:~/practica3$
```

Figura 6 — Ejecución del programa refactorizado con los datos de ejemplo.

Tests finales (todos en verde):

```
alejandro@ubuntu: ~/practica3

alejandro@ubuntu:~/practica3$ java -cp lib/junit.jar:lib/hamcrest.jar:out/main:out/test \
      org.junit.runner.JUnitCore TarifaConciertoTest RegresionFacturaTest
JUnit version 4.13.2
.........
Time: 0,083

OK (9 tests)

alejandro@ubuntu:~/practica3$
```

Figura 7 — Los 9 tests pasan correctamente tras todos los refactorings.

Commit final:

```
git commit -m "chore: salida verificada y limpieza final"
```

9. Conclusiones

Esta práctica me ha resultado bastante más interesante que las anteriores. Al principio pensaba que refactoring era solo "ordenar el código" y cambiarle el nombre a las variables, pero ahora entiendo que es mucho más que eso: se trata de cambiar la *estructura* del código sin cambiar lo que hace, y para eso necesitas los tests como red de seguridad.

El patrón Strategy me ha parecido muy útil. El hecho de que añadir un tipo nuevo de concierto ahora solo requiera una clase nueva, sin tocar nada más, es una mejora muy concreta y visible.

Lo que más me ha costado ha sido mantener los tests en verde en todo momento mientras hacía los cambios. Hubo un momento en el que cambié la fórmula de créditos de HEAVY y el test de regresión falló porque el total no cuadraba. Gracias al test lo pillé rápido, si no me hubiese dado cuenta mucho más tarde.

Con Git también he practicado bastante. Al principio me olvidé de crear la rama antes de empezar y tuve que reorganizar los commits, pero al final el historial quedó bastante limpio.

10. Bibliografía

- Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2.ª ed.). Addison-Wesley Professional.
- Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.
- Chacon, S., y Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. Recuperado de https://git-scm.com/book/es
- Oracle Corporation. (2023). *Java SE 21 — Interface documentation*. Recuperado de https://docs.oracle.com/en/java/javase/21/
- Refactoring Guru. (s.f.). *Strategy pattern*. Recuperado de https://refactoring.guru/es/design-patterns/strategy